CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 7: Authentication and Middleware Pipeline

CO/PO	CO2, CO6 / PO3, PO5
Objective	To implement JWT-based authentication and input validation as part of the Express middleware pipeline.
Prerequisites	• Practical 6 completed working full-stack Task Management application • Basic understanding of hashing and tokens (conceptual, covered in self-study material)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 7 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 4 (authentication and authorization, JWT token generation, error handling middleware, request validation and sanitization).
Architecture / Diagram	

Diagram	`POST /register  → hash password → save User → 201`
	`POST /login  → verify password → sign JWT → return token`
	
	`Protected Route Flow:`
	`Client Request (Authorization: Bearer <token>)`
	`  |`
	`  v`
	`[Auth Middleware] → verifies token → req.user set`
	`  |`
	`  v`
	`[Validation Middleware] → checks input`
	`  |`
	`  v`
	`Controller (task routes)`

Problem Definition	
Problem Statement	• Add user registration and login functionality to the Task Management application. • Hash passwords using bcrypt before saving the User document to MongoDB. • Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour). • Protect all task routes using an authentication middleware that verifies the JWT. • Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.
Key Questions / Analysis	• Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project? • What does the authentication middleware actually verify, and what happens if the token is missing or expired? • Why should input validation happen on the server even if the frontend already validates the same fields?
Supplementary Problems	• Add a /me endpoint that returns the currently logged-in user's details using the decoded JWT. • Implement token expiry handling on the frontend redirect to login if a protected request returns 401.

	• Add a logout mechanism that clears the token from the client.
Key Skills Addressed	• Hashing passwords securely using bcrypt • Generating and verifying JWT tokens • Writing authentication middleware to protect routes • Implementing server-side input validation and sanitization
Learning Outcome	Students will be able to implement a complete authentication flow using JWT, protect backend routes with middleware, and enforce server-side input validation.
Lab Session Step-by-Step	

Steps	1. Install required packages:
	`npm install bcryptjs jsonwebtoken`
	2. Create a User schema with email and hashed password fields.
	3. Implement the register route hash the password before saving:
	`const hashedPassword = await bcrypt.hash(password, 10);`
	`const user = await User.create({ email, password: hashedPassword });`
	4. Implement the login route compare password and sign a JWT:
	`const isMatch = await bcrypt.compare(password, user.password);`
	`const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });`
	5. Create an auth middleware that verifies the token from the Authorization header:
	`const token = req.headers.authorization?.split(' ')[1];`
	`const decoded = jwt.verify(token, process.env.JWT_SECRET);`
	`req.user = decoded;`
	6. Apply the auth middleware to all task routes.
	7. Add a validation middleware that checks required fields exist before reaching the controller.
	8. Test the full flow in Postman: register → login → copy token → access protected task routes.

Tools / Technology / Resources Required	• Node.js (v18+), npm • bcryptjs, jsonwebtoken • Express.js, Mongoose • Postman (for token-based testing)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Walk through why bcrypt and not plain SHA hashing is used bcrypt is intentionally slow and salts automatically, which matters for security. • Demonstrate testing protected routes in Postman using the Authorization header with Bearer <token> before students attempt the frontend. • Emphasize JWT_SECRET must come from .env, never hardcoded connect this back to the secure storage principle in the syllabus. • Reserve time to show what happens with an expired or tampered token this builds real understanding of why verification matters.
Common Mistakes	• Storing passwords as plain text or using a reversible encryption instead of bcrypt hashing. • Forgetting to send the Authorization header when testing protected routes, then assuming the middleware is broken. • Hardcoding the JWT secret directly in code instead of using an environment variable. • Not handling the case where jwt.verify() throws on an invalid/expired token, crashing the server.

Troubleshooting Guide

Symptom	Likely Cause	Fix
401 Unauthorized on every request	Authorization header missing or malformed in Postman/client	Set header as Authorization: Bearer <token> exactly, including the space after Bearer
Server crashes on invalid token	jwt.verify() not wrapped in try/catch	Wrap token verification in try/catch and return 401 on failure instead of letting it throw uncaught
Login always fails even with correct password	Comparing plain password to hashed password directly instead of using bcrypt.compare	Use bcrypt.compare(plainPassword, hashedPassword) rather than ===
JWT_SECRET is undefined	.env file not loaded or variable name mismatch	Confirm dotenv.config() runs before JWT_SECRET is used and the .env key name matches exactly

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a /me endpoint that returns the logged-in user's details from the decoded token. • Implement basic frontend handling for 401 responses (redirect to a login prompt).

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 6, updated with new commits • Working register/login flow with JWT-protected task routes • .env excluded via .gitignore; JWT_SECRET never committed • At least one commit specifically for authentication implementation

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**